

Reviewer Pack — Fourier Coefficient Sum and Disjointness Communication Compl...

Entry d551d2a8e9bf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 21:10:41 UTC

Fourier Coefficient Sum and Disjointness Communication Complexity

Entry ID: d551d2a8e9bf **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 21:10:41 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis **Field B** (complexity object): Communication Complexity of Disjointness Problem

Statement:

For a boolean function $f: \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$ representing the disjointness problem, the communication complexity $CC(f)$ is $\Omega(\sqrt{\sum |F_k|})$, where F_k are the Fourier coefficients of f .

Rationale (proposer's reasoning):

Fourier coefficients capture the influence of variables, which relates to the structure of the function. Disjointness communication complexity is known to be high, so linking it to the sum of Fourier coefficients could reveal structural properties.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7344a653f2115a4f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def fast_walsh_hadamard_transform(x):
    n = len(x)
    if n == 1:
        return x
    even = fast_walsh_hadamard_transform(x[0::2])
    odd = fast_walsh_hadamard_transform(x[1::2])
    result = [0] * n
    for k in range(n // 2):
        result[k] = even[k] + odd[k]
        result[k + n // 2] = even[k] - odd[k]
    return result

def fourier_coefficients(f, n):
    def f_hat(k):
        sum_val = 0
        for x in range(2 ** n):
            sum_val += f(x) * math.cos(math.pi * k * x / (2 ** n))
        return sum_val / (2 ** n)
    return [f_hat(k) for k in range(2 ** n)]

def disjointness_communication_complexity(n):
    def f(x, y):
        return int(sum(xi != yi for xi, yi in zip(x, y)) > 0)

    coefficients = fourier_coefficients(f, n)
    sum_abs_coeffs = sum(abs(coeff) for coeff in coefficients)
    return math.sqrt(sum_abs_coeffs)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    cc = disjointness_communication_complexity(n)
    instances_tested = 1
    conjecture_holds = False
    counterexample = ""

    return {
```

```

        "metric_name": "disjointness_communication_complexity",
        "metric_value": cc,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_cc = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_cc) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_cc} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1113c3.py", line 68, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1113c3.py", line 49, in run_trial
    cc = disjointness_communication_complexity(n)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1113c3.py", line 42, in disjointness_communication_complexity
    coefficients = fourier_coefficients(f, n)
                  ^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1113c3.py", line 36, in fourier_coefficients
    return [f_hat(k) for k in range(2 ** n)]
           ^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1113c3.py", line 34, in f_hat
    sum_val += f(x) * math.cos(math.pi * k * x / (2 ** n))
              ^^^^^

```

TypeError: disjointness_communication_complexity.<locals>.f() missing 1 required positional argument

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to missing argument 'y' in the disjointness function, preventing any data collection. | next: Fix the function signature to include both x and y arguments for the disjointness predicate

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	137967
2	propose	ollama_remote	qwen3:8b	0	0	139795
3	preregistration	ollama_remote	qwen3:8b	0	0	32027
4	novelty	ollama_remote	qwen3:8b	0	0	27225
5	novelty	ollama_remote	qwen3:8b	0	0	24724
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13190
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9228
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8689
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9392
10	judge	ollama_remote	qwen3:8b	0	0	29342

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 431579 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/d551d2a8e9bf.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d551d2a8e9bf.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d551d2a8e9bf.tar.gz (if generated)